Considerations when implementing CI/CD for Microsoft Fabric

Kevin Chant



Microsoft Fabric



Agenda



1. Bio
2. Why perform CI/CD in Fabric.
3. Microsoft Fabric version control.
4. Testing strategies.
5. CI maturity levels.
6. Deployment strategies.

Kevin Chant



- Lead Technology Advocate in the Netherlands
- Worked in IT since the days of Windows 95
- Experience in various sectors
- Various certifications, Data Platform MVP



- Twitter/Blue Sky: @kevchant
- LI: https://www.linkedin.com/in/kevin-chant/
- Blog: https://www.KevinRChant.com
- GitHub: https://github.com/kevchant





Why perform CI/CD in Microsoft Fabric?



- Continuous Integration to merge updates frequently. Ideally testing changes to leave everything in a workable state.
- Continuous delivery to ensure updates can be deployed to other environments in a fast and repeatable manner.
- Continuous deployment when automatic rollout of deployment is required.

Benefits of CI/CD in Microsoft Fabric?



- Faster release cycle.
- Audit trail of changes.
- Possibility of automated tests at early stage.
- Reliable and more consistent deployments.
- Allows working with new industry innovations.

Microsoft Fabric Git integration



- Version control for Fabric items.
- Allows metadata for supported items in a workspace to be synchronized with a Git repository.
  - To be more precise a workspace synchronizes with a branch.
- Requires Fabric or Power BI Premium capacity.
- Items supported at various levels.
- Supports cloud-based versions of Azure DevOps and GitHub.

Example of supported items

- Data pipelines (preview)
- Dataflows gen2 (preview)
- Eventhouse and KQL database (preview)
- EventStream (preview)
- Lakehouse (preview)
- Mirrored database (preview)
- Notebooks
- Paginated reports (preview)
- Reflex (preview)
- Warehouses (preview)
- Reports (except reports connected to semantic models hosted in Azure Analysis Services, SQL Server Analysis Services, or reports exported by Power BI Desktop that depend on semantic models hosted in MyWorkspace) (preview)
- Semantic models (except push datasets, live connections to Analysis Services, model v1) (preview)
- Spark Job Definitions (preview)
- Spark environment (preview)
- SQL database (preview)

Benefits of Microsoft Fabric Git integration

- Allows you to track and manage changes to Fabric items.
- Reduces conflicts and allows history of changes.
- Based on extremely popular Git version control system.
- Enables you to perform CI/CD with multiple Fabric items collectively.

Git integration features

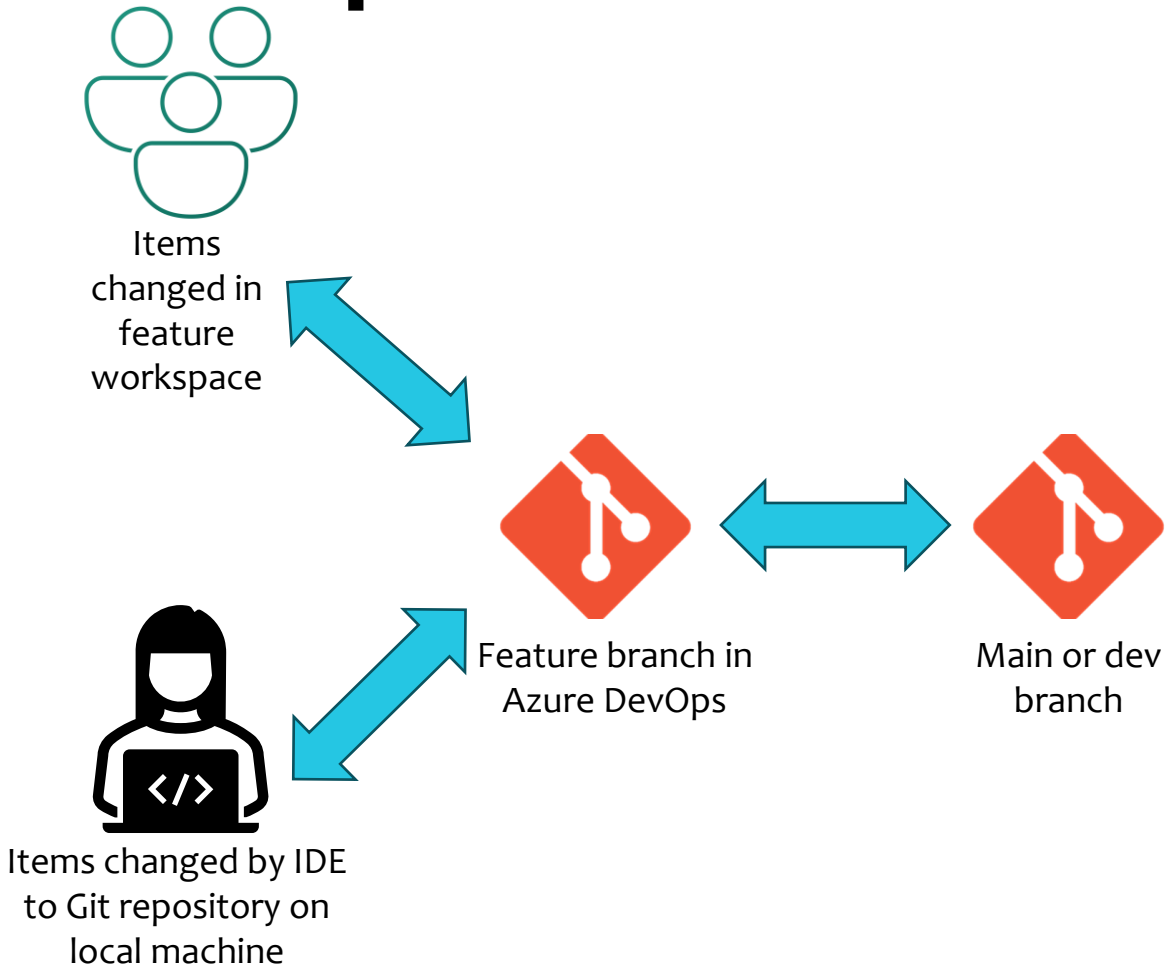


- Can change items in workspace and commit to repository.
- Can also update from repository.
- Can branch out to new workspace to create “feature” workspaces.
- Demo

Considerations for Azure DevOps or GitHub

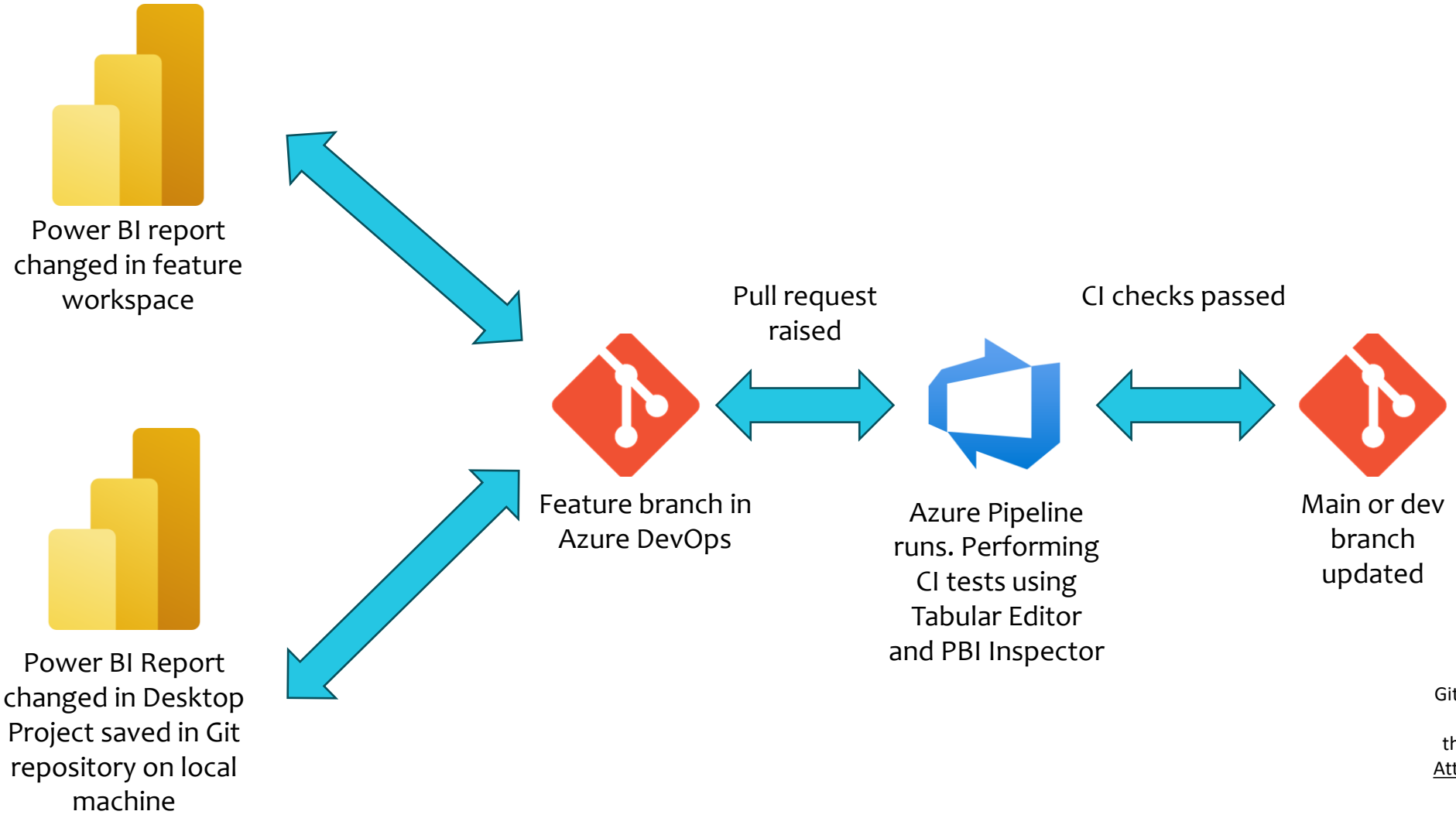
- Largely depends on what is currently used in environment.
- Azure DevOps uses Entra authentication. GitHub currently uses PAT’s.
- Azure DevOps can be region specific. GitHub now allows you to request to deploy organization specifically in the EU, Australia and US..
- If new adoption, consider learning curve.

Using Git integration with recommended development process



Git Logo by Jason Long is licensed under the Creative Commons Attribution 3.0 Unported License.

Suggest implement unit/integration tests



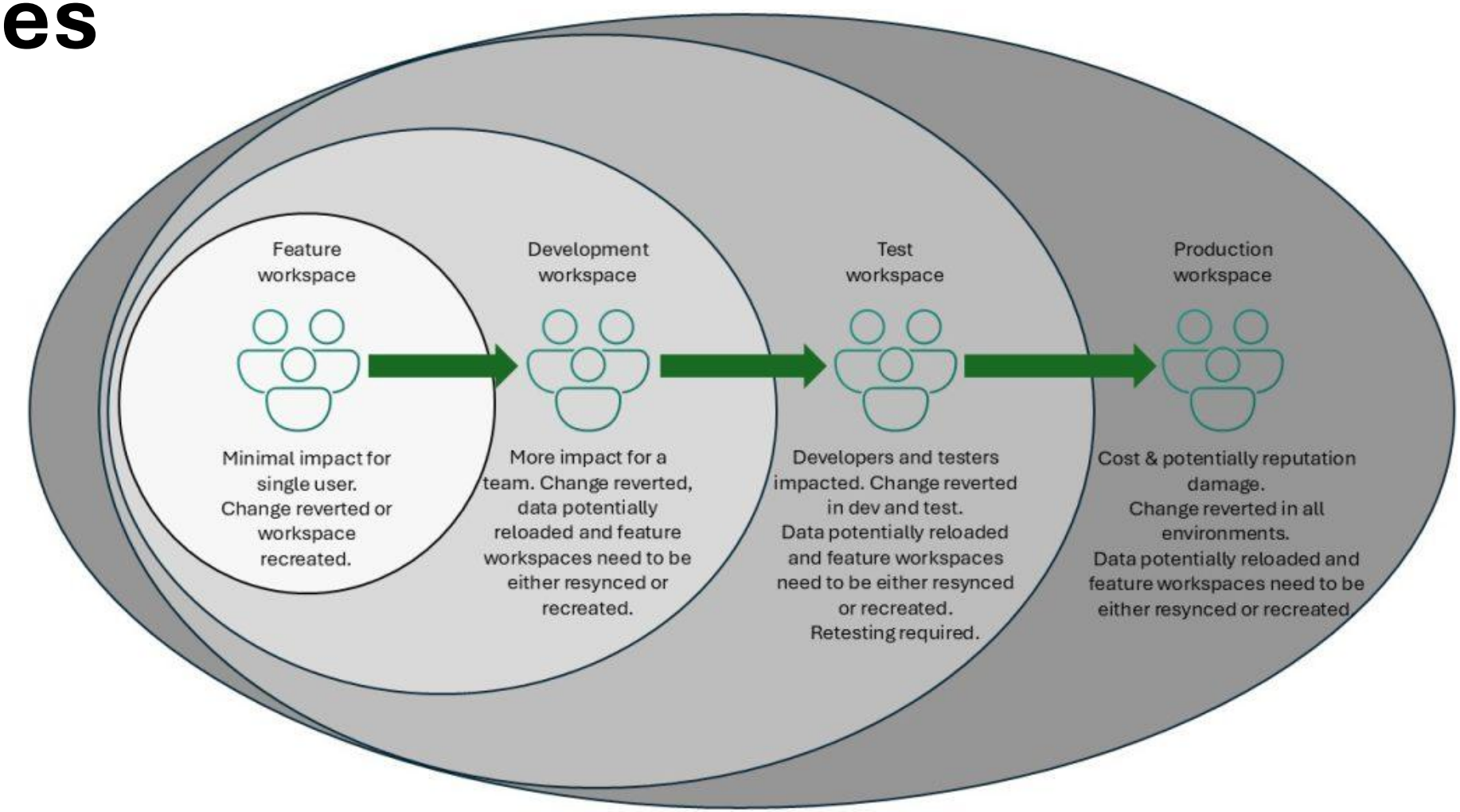
Git Logo by Jason Long is licensed under the Creative Commons Attribution 3.0 Unported License.

Testing strategies for items

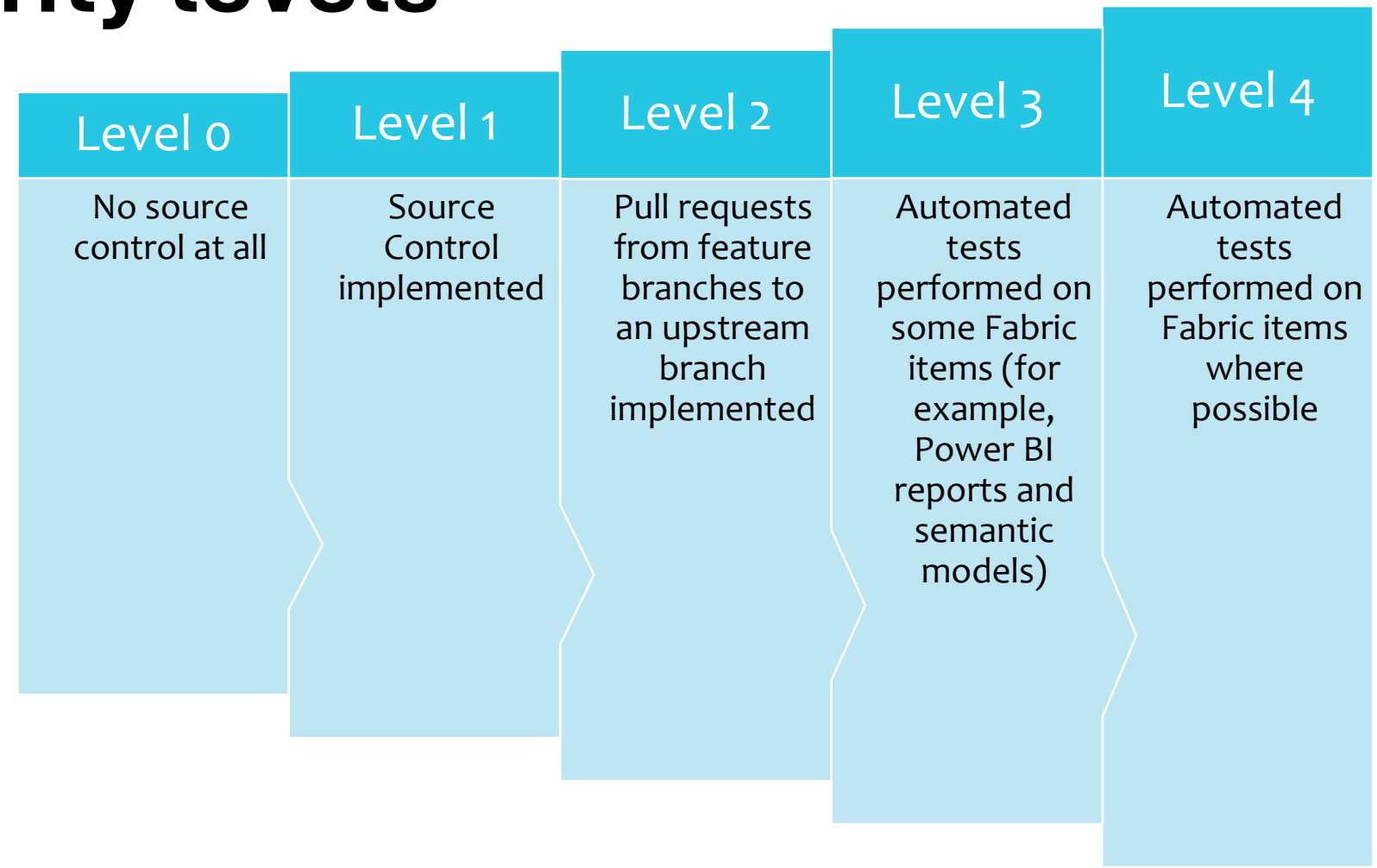


- Power BI reports – PBI Inspector (V2 supports other Fabric items)
- Semantic models – Tabular Editor or semantic link
- Data Pipelines – Data Factory Testing Framework
- Lakehouses, Data Warehouses & KQL Databases – Great expectations, pytest, etc.
- SQL database in Fabric – SQL code analysis.

Impact of changes identified at different stages



Microsoft Fabric Continuous Integration maturity levels



Deployment strategies

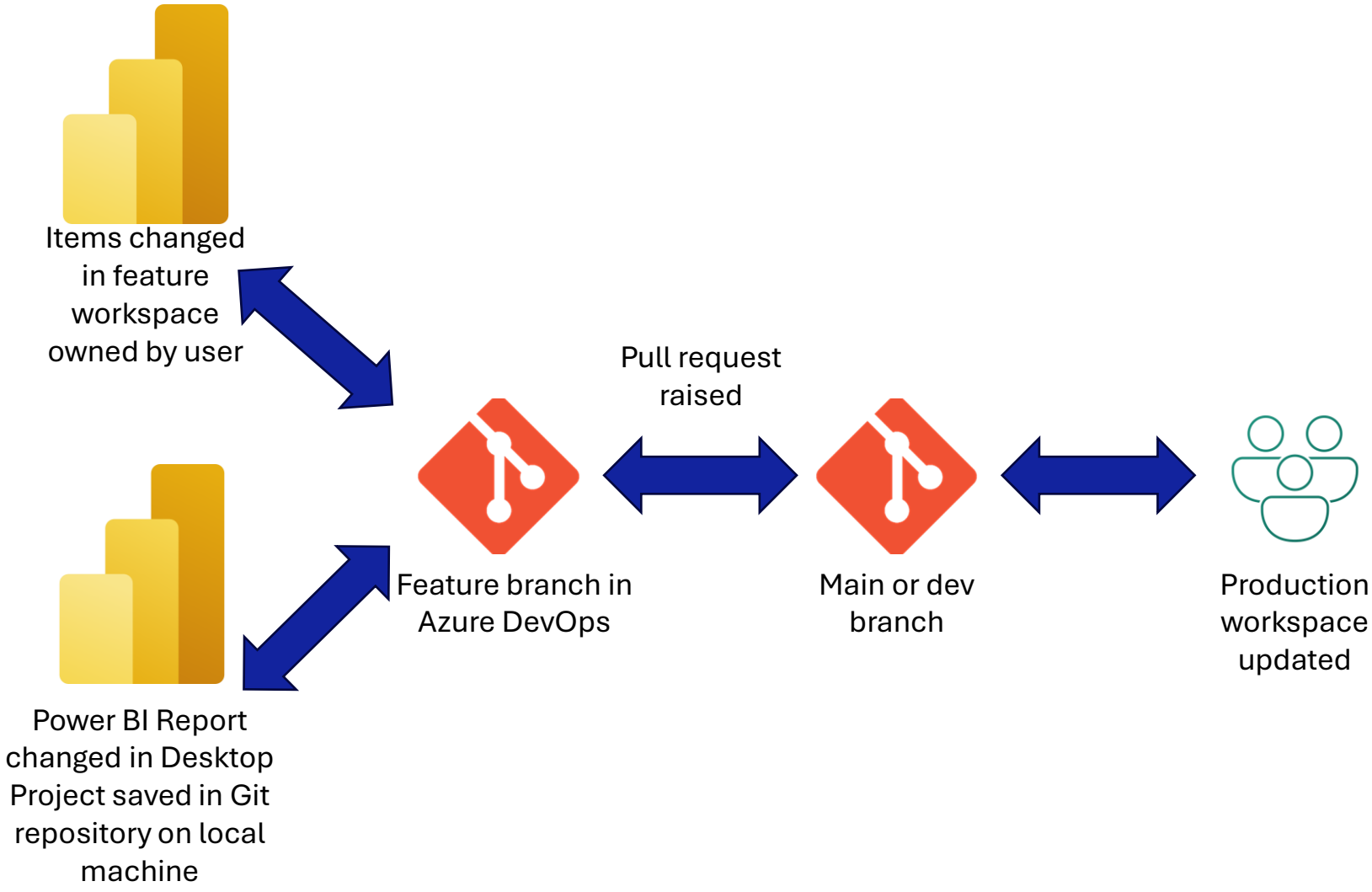


- Microsoft released article last September.
- Includes previously shown recommended development process (CI) and also covers release options (CD).
- I enriched these with additional considerations for deployment strategies.

Option 1 – Git-based deployments

- Have workspaces connected to different branches in the same Git repository.
- Merge updates to different workspaces via pull requests.
- Can be done manually or with Fabric Git APIs supplemented by other APIs.

Option 1 diagram



Git Logo by Jason Long is licensed under the Creative Commons Attribution 3.0 Unported License.

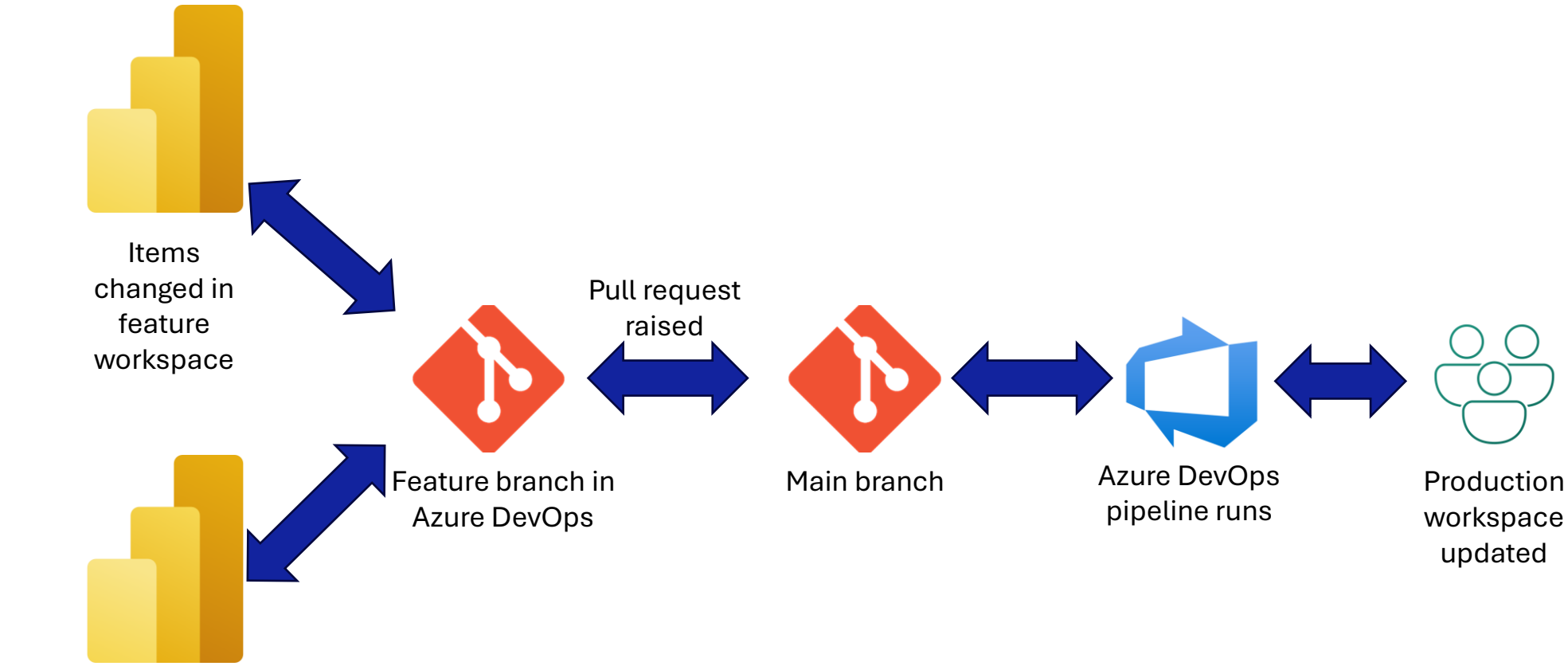
Option 1 – Guidance

- Git knowledge required.
- Ideal for scenarios that require only minor source changes.
- Recommend automating this process.

Option 2 – Git-based deployments using build pipeline

- Deploy to different workspaces through a suitable pipeline deployment service (Azure Pipelines, GitHub Actions).
- Typically done with either APIs or a layer of abstraction.
- Recommendation is that workflow for each stage contains:
  - Build for unit tests
  - Release to perform update

Option 2 diagram



Git Logo by Jason Long is licensed under the Creative Commons Attribution 3.0 Unported License.

Option 2 – Guidance

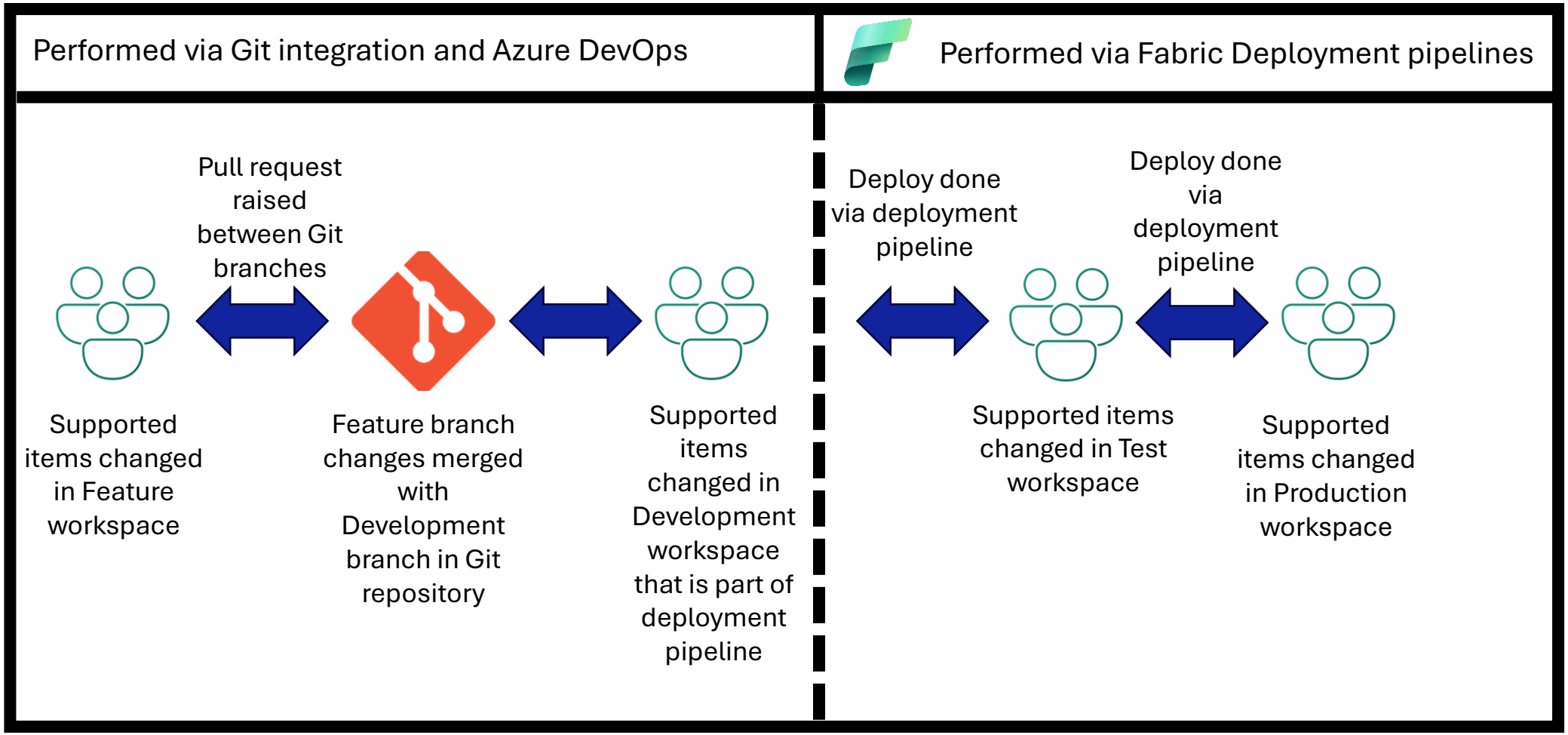
- Knowledge of ALM service required (Azure DevOps, GitHub).
- Ideal for more complex scenarios and when you implement DataOps.
- Suggest looking at tooling as layer of abstraction such as:
  - PowerShell modules.
  - Fabric-cicd Python library.
  - Fabric Command Line Interface.

Option 3 – Microsoft Fabric deployment pipelines

- Use Fabric deployment pipeline to update a workspace that represents development environment/stage.

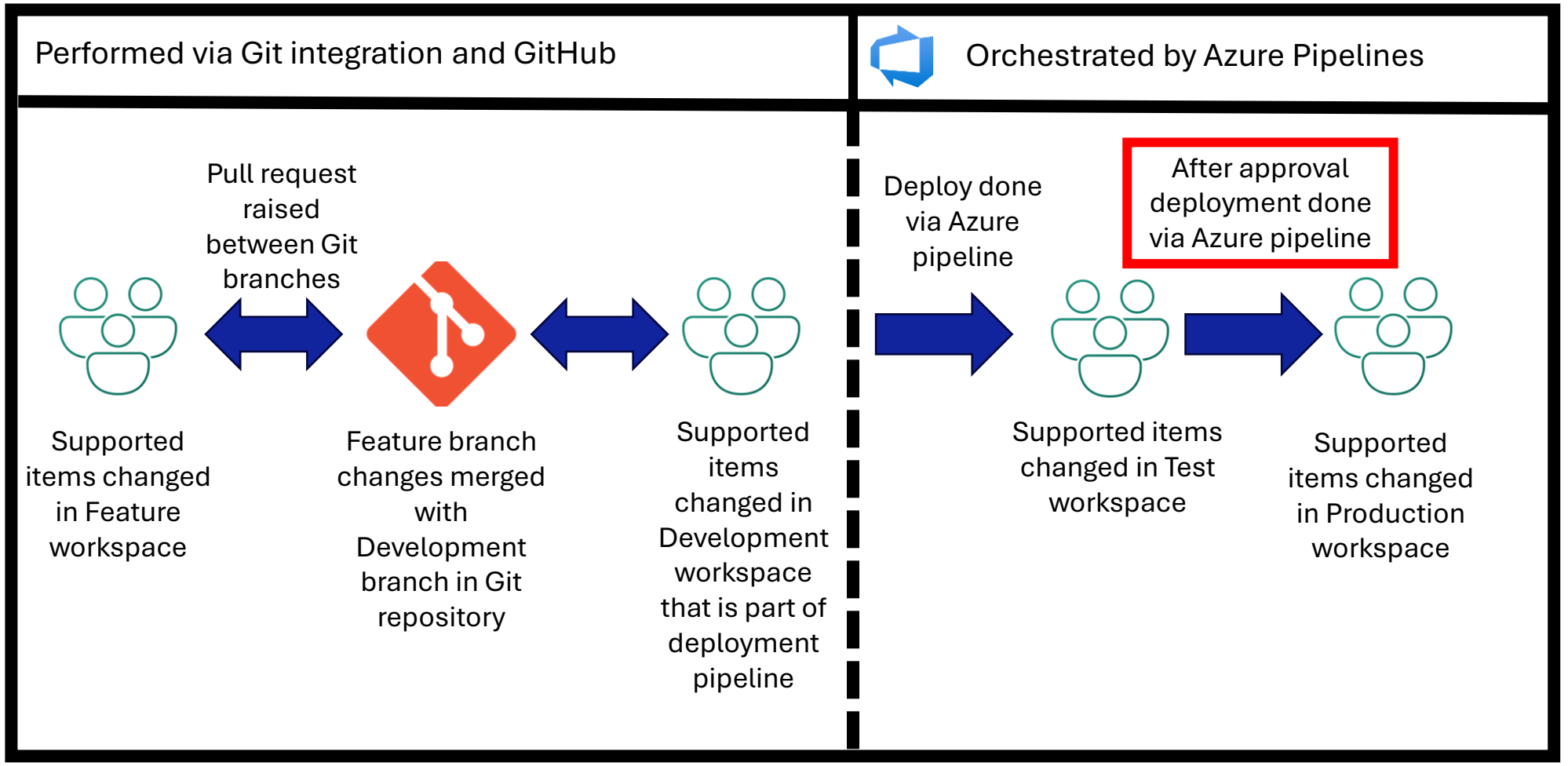
- From there, orchestrate with Microsoft Fabric deployment pipelines.

Option 3 with deployment pipelines



Git Logo by Jason Long is licensed under the Creative Commons Attribution 3.0 Unported License.

Deployment pipelines orchestrated by Azure Pipelines



Git Logo by Jason Long is licensed under the Creative Commons Attribution 3.0 Unported License.

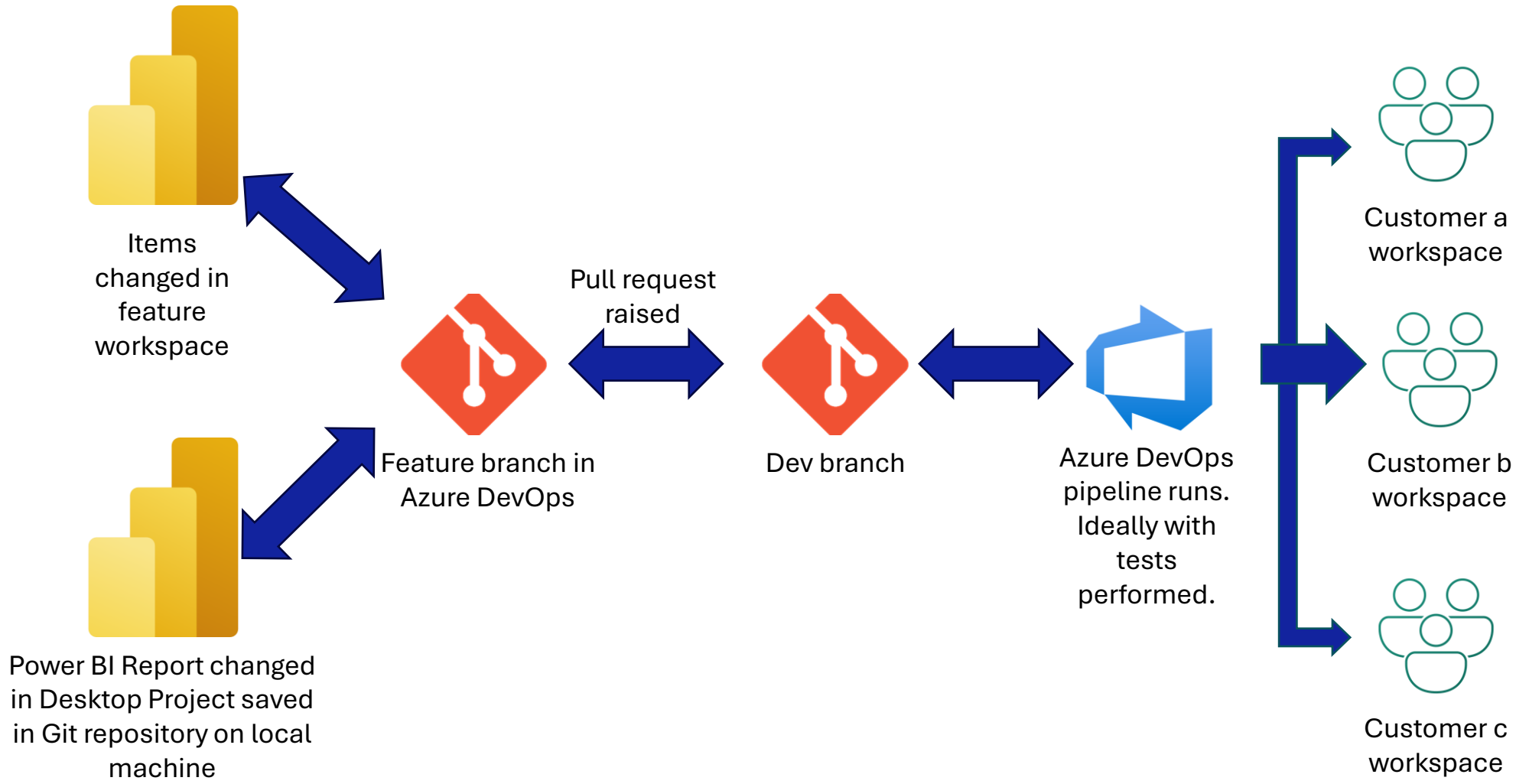
Option 3 – Guidance

- Ideal for business citizens.
- Can also apply various deployment rules, such as data sources.
- When requirements for existing pipeline go beyond capabilities can look to orchestrate existing stages in ALM offering.

Option 4 – For multiple clients/solutions/tenants

- Similar to option 2.
- All development and test done in own tenant.
- Then deployed to workspaces in other tenants via your ALM of choice.

Option 4 diagram



Git Logo by Jason Long is licensed under the Creative Commons Attribution 3.0 Unported License.

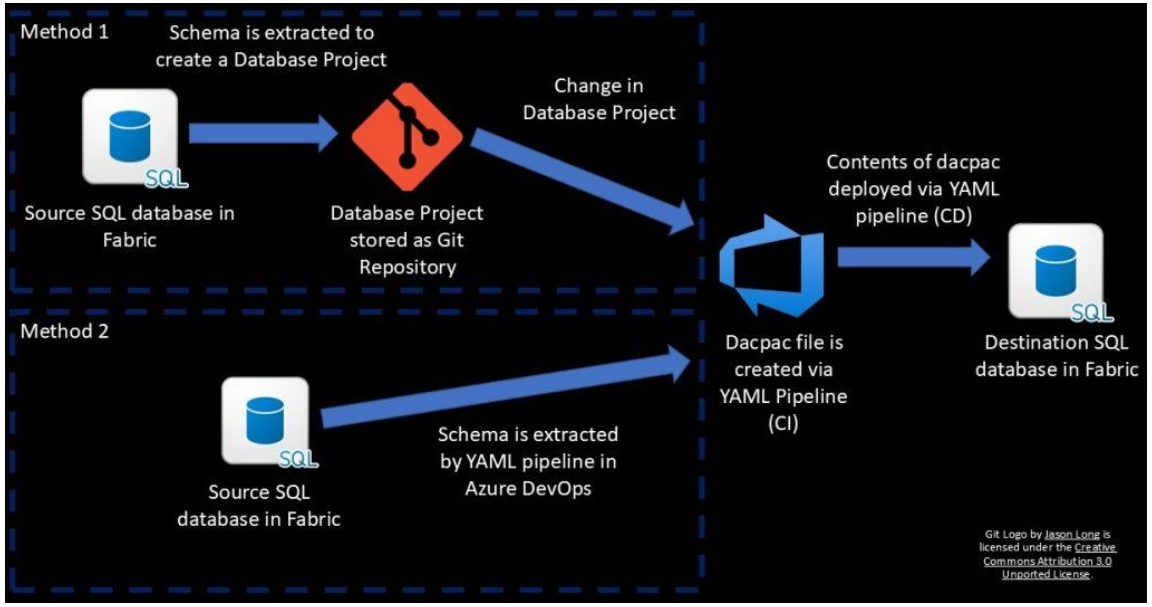
Option 4 – Guidance

- Aimed mostly at ISV's. However, useful for custom reports as well.
- If required, recommend implementing option 2 internally for consistency.
- Ask if this deployment is the right option or would hosting a Fabric Org App internally or offering a custom workload be a better option?

Fabric Data Warehouses and SQL database in Fabric

- Can work with majority of previous options.

- Traditional Database Project and dacpac method still an option.



Considerations for developer machines

- In order to efficiently work with Fabric items locally:
  - Git
  - Visual Studio. Recommend GitHub Copilot to increase productivity.

- For Power BI reports:
  - Power BI Desktop

To summarize



- For best coverage of items implement Git integration.
- Various Fabric items can be tested at early stage.
- Strive to improve Continuous Integration maturity level.
- Various release options to cater for multiple needs.
- Traditional SQL CI/CD still an option.
- Remember to take local applications into consideration.

Questions



Thank you



- Twitter/Bluesky: @kevchant
- LI: https://www.linkedin.com/in/kevin-chant/
- Blog: https://www.KevinRChant.com
- GitHub: https://github.com/kevchant